

Homework 5

Haozheng Wang

December 2025

1 Define Classifiers

1.1 Linear classifier

For visualization results of a linear classifier, please refer to 3.3.

```
1 class LinearClassifier(nn.Module):
2     def __init__(self, in_channels: int, out_channels: int):
3         super().__init__()
4         self.fc = nn.Linear(in_channels, out_channels)
5
6     def forward(self, x: torch.Tensor):
7         x = x.view(x.size(0), -1)
8         x = self.fc(x)
9         return x
```

1.2 Full-connected neural network classifier

We made several improvements to barebones FCNN, and achieved an accuracy above 60% in 60 epochs without using CNN. The network we adopted has 1 input layer and 3 hidden layers featuring the following structure: *-[Linear-BatchNorm1d-ReLU-Dropout]-*, with the number of neurons of each layer decreasing rather than remaining constant. For visualization results of this fine-tuned FCNN, please refer to 3.3.

```
1 class FCNN(nn.Module):
2     def __init__(self, in_channels: int, hidden_channels: int,
3         out_channels: int):
4         super().__init__()
5         self.network = nn.Sequential(
6             nn.Linear(in_features=32 * 32 * 3, out_features=2048),
7             nn.BatchNorm1d(num_features=2048),
8             nn.ReLU(inplace=True),
9             nn.Dropout(p=0.5),
10
11             nn.Linear(in_features=2048, out_features=1024),
12             nn.BatchNorm1d(num_features=1024),
13             nn.ReLU(inplace=True),
```

```

13         nn.Dropout(p=0.5),
14
15         nn.Linear(in_features=1024, out_features=512),
16         nn.BatchNorm1d(num_features=512),
17         nn.ReLU(inplace=True),
18         nn.Dropout(p=0.5),
19
20         nn.Linear(in_features=512, out_features=256),
21         nn.BatchNorm1d(num_features=256),
22         nn.ReLU(inplace=True),
23         nn.Dropout(p=0.5),
24
25         nn.Linear(in_features=256, out_features=10)
26     )
27
28     def forward(self, x: torch.Tensor):
29         x = x.view(x.size(0), -1)
30         return self.network(x)

```

2 Implement the training and testing function

In order to enhance the generalization ability of our neural networks, we carry out some data augmentation by random cropping and random horizontal flipping. They turn out to be enough for preventing overfitting, but it should not be ignored that too much augmentation may lead to learning difficulties, resulting in poor performance on both the training and validation sets.

```

1 transform_train = transforms.Compose([
2     transforms.RandomCrop(32, padding=4),
3     transforms.RandomHorizontalFlip(p=0.5),
4     transforms.ToTensor(),
5     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
6 ])

```

The complete code of the training function are shown below. TensorBoard's APIs are called to record and visualize the training results. For simplification, we use the test set as our validation set during the training process.

```

1 def train(model, optimizer, scheduler, args):
2     writer = SummaryWriter(log_dir=f'runs/{args.model}-{args.
3         optimizer}-{args.scheduler}')
4     criterion = nn.CrossEntropyLoss()
5     transform_train = transforms.Compose([
6         transforms.RandomCrop(32, padding=4),
7         transforms.RandomHorizontalFlip(p=0.5),
8         transforms.ToTensor(),
9         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
10    ])
11     transform_val = transforms.Compose([

```

```

11         transforms.ToTensor(),
12         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
13     ])
14     batch_size = 128
15     # create dataset
16     trainset = torchvision.datasets.CIFAR10(root='./data', train=
17         True, download=True, transform=transform_train)
18     valset = torchvision.datasets.CIFAR10(root='./data', train=
19         False, download=True, transform=transform_val)
20     # create dataloader
21     trainloader = torch.utils.data.DataLoader(trainset, batch_size=
22         batch_size, shuffle=True, num_workers=2)
23     valloader = torch.utils.data.DataLoader(valset, batch_size=
24         batch_size, shuffle=True, num_workers=2)
25
26     best_val_accuracy = 0.0
27     for epoch in range(60):
28         # train
29         model.train()
30         epoch_train_loss = 0.0
31         train_correct = 0
32         train_total = 0
33         for i, data in enumerate(trainloader, 0):
34             # get the inputs; data is a list of [inputs, labels]
35             inputs, labels = data
36             # zero the parameter gradients
37             optimizer.zero_grad()
38             # forward
39             outputs = model(inputs)
40             loss = criterion(outputs, labels)
41             # loss backward
42             loss.backward()
43             # optimize
44             optimizer.step()
45             # print statistics
46             epoch_train_loss += loss.item()
47             _, predicted = torch.max(outputs.data, 1)
48             train_total += labels.size(0)
49             train_correct += (predicted == labels).sum().item()
50         avg_epoch_train_loss = epoch_train_loss / len(trainloader)
51         train_accuracy = 100 * train_correct / train_total
52         writer.add_scalar('Training loss (per epoch)',
53             avg_epoch_train_loss, epoch)
54         writer.add_scalar('Training Accuracy (per epoch)',
55             train_accuracy, epoch)
56         # adjust learning rate
57         scheduler.step()
58
59         # test
60         model.eval()
61         val_correct = 0
62         val_total = 0
63         with torch.no_grad():
64             for data in valloader:
65                 # forward
66                 inputs, labels = data
67                 outputs = model(inputs)

```

```

62         # calculate accuracy
63         _, predicted = torch.max(outputs.data, 1)
64         val_total += labels.size(0)
65         val_correct += (predicted == labels).sum().item()
66     val_accuracy = 100 * val_correct / val_total
67     writer.add_scalar('Validation Accuracy', val_accuracy,
68                       epoch)
69     # save checkpoint
70     if val_accuracy > best_val_accuracy:
71         best_val_accuracy = val_accuracy
72         checkpoint = {
73             'epoch': epoch,
74             'model_state_dict': model.state_dict(),
75             'optimizer_state_dict': optimizer.state_dict(),
76             'accuracy': val_accuracy,
77             'args': vars(args)
78         }
79         torch.save(checkpoint, 'checkpoint_fcnn.pth')
80     print(f'Best Validation Accuracy: {best_val_accuracy:.2f}')
81     writer.close()

```

The implementation of the testing function are shown below.

```

1 def test(model, args):
2     # load checkpoint
3     checkpoint = torch.load('checkpoint_fcnn.pth', weights_only=
4                               True)
5     model.load_state_dict(checkpoint['model_state_dict'])
6     # create testing dataset
7     transform = transforms.Compose(
8         [transforms.ToTensor(),
9          transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
10    testset = torchvision.datasets.CIFAR10(root='./data', train=
11        False, download=True, transform=transform)
12    # create dataloader
13    testloader = torch.utils.data.DataLoader(testset, batch_size
14        =256, shuffle=False, num_workers=2)
15    # test
16    model.eval()
17    test_correct = 0
18    test_total = 0
19    with torch.no_grad():
20        for data in testloader:
21            # forward
22            inputs, labels = data
23            outputs = model(inputs)
24            # calculate accuracy
25            _, predicted = torch.max(outputs.data, 1)
26            test_total += labels.size(0)
27            test_correct += (predicted == labels).sum().item()
28    test_accuracy = 100 * test_correct / test_total
29    print(f'Test Accuracy: {test_accuracy} %')

```

3 Report

3.1 Compare AdamW and SGD optimizer

For the sake of rigor, we conducted two sets of experiments in this section, one under CosineAnnealingLR condition and the other under StepLR condition.

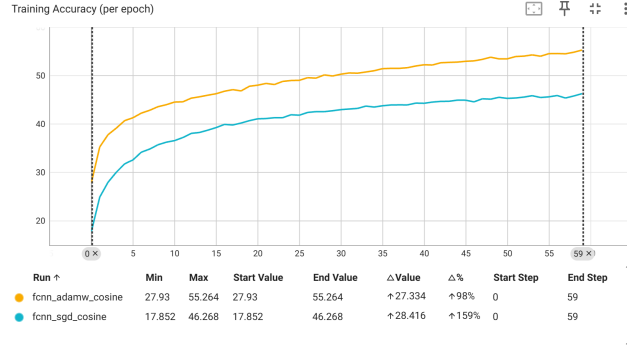
The parameter configuration for the first experiment is as follows:

- optimizer: AdamW / SGD
- AdamW_lr=0.001 / SGD_lr=0.001, SGD_momentum=0.9
- epoch_num=60
- batch_size=128
- scheduler: CosineAnnealingLR
- CosineAnnealingLR_T_max=100

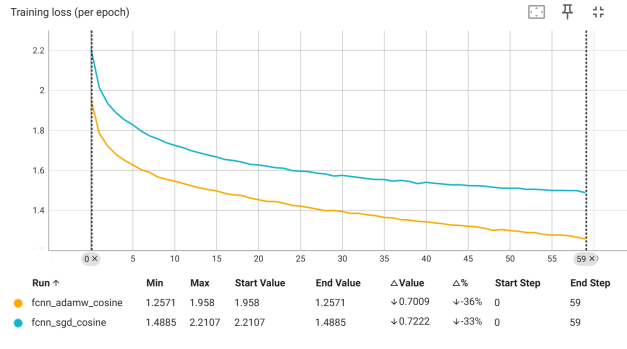
The parameter configuration for the second experiment is as follows:

- optimizer: AdamW / SGD
- AdamW_lr=0.001 / SGD_lr=0.001, SGD_momentum=0.9
- epoch_num=60
- batch_size=128
- scheduler: StepLR
- StepLR_step_size=10

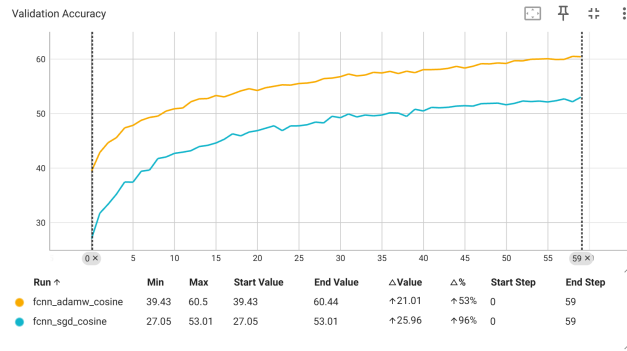
The visualization results are displayed in the following two pages. According to the visualized curves, the training processes using these two optimizers are generally parallel, but AdamW performs significantly better on all metrics. It achieves higher accuracy on both training data and test data, and converges to a smaller loss as well. These results are not so surprising, since AdamW is expected to be a more powerful optimizer in most aspects.



(a) Training Accuracy

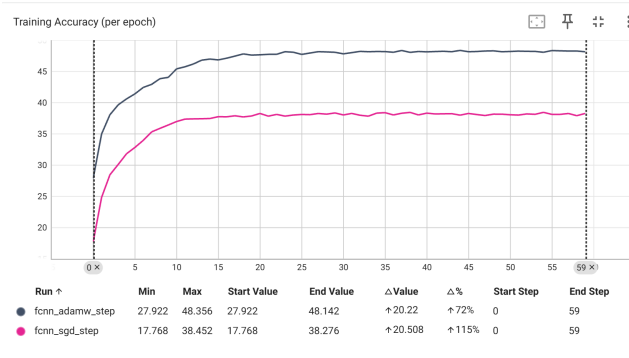


(b) Training Loss

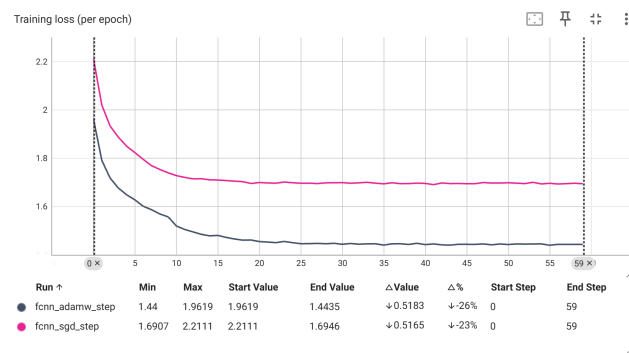


(c) Validation Accuracy

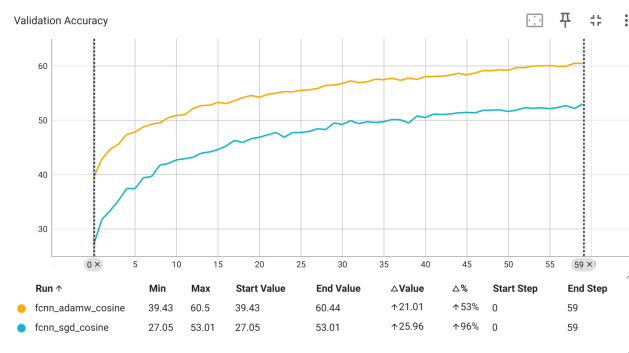
Figure 1: AdamW vs SGD (CosineAnnealingLR)



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 2: AdamW vs SGD (StepLR)

3.2 Compare StepLR and CosineAnnealingLR scheduler

Again, we conducted two sets of experiments in this section for the sake of rigor, one under AdamW condition and the other under SGD condition.

The parameter configuration for the first experiment is as follows:

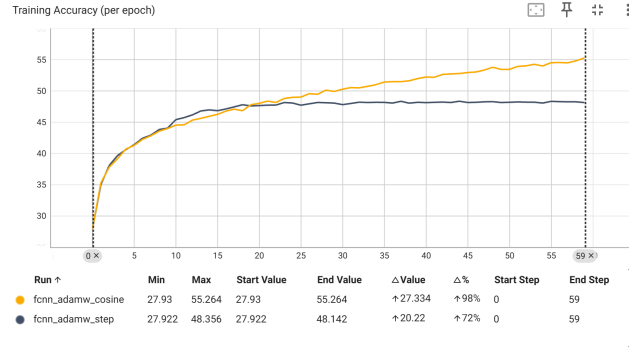
- scheduler: CosineAnnealingLR / StepLR
- CosineAnnealingLR_T_max=100 / StepLR_step_size=10
- epoch_num=60
- batch_size=128
- optimizer: AdamW
- AdamW_lr=0.001

The parameter configuration for the second experiment is as follows:

- scheduler: CosineAnnealingLR / StepLR
- CosineAnnealingLR_T_max=100 / StepLR_step_size=10
- epoch_num=60
- batch_size=128
- optimizer: SGD
- SGD_lr=0.001, SGD_momentum=0.9

The visualization results are displayed in the following two pages. According to the visualized curves, StepLR performs slightly better at the beginning stage, but soon it is surpassed by CosineAnnealingLR by all metrics.

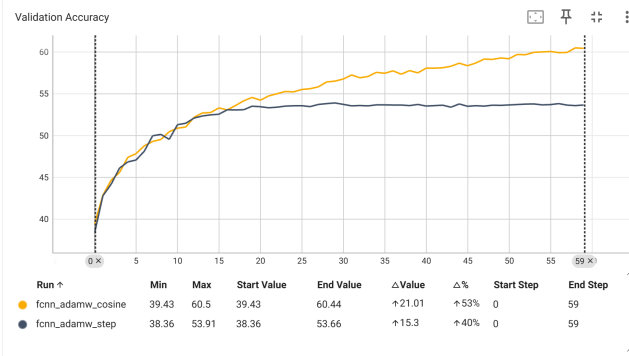
It is likely that multiplying the learning rate by a constant after each epoch in StepLR is too sharp, which leads to the model's reaching performance bottleneck quickly (converging in less than 20 epochs); Meanwhile, the cosine scheduler effectively smooth the adjustment of the learning rate, and the model still showed performance improvement even after 40 epochs.



(a) Training Accuracy

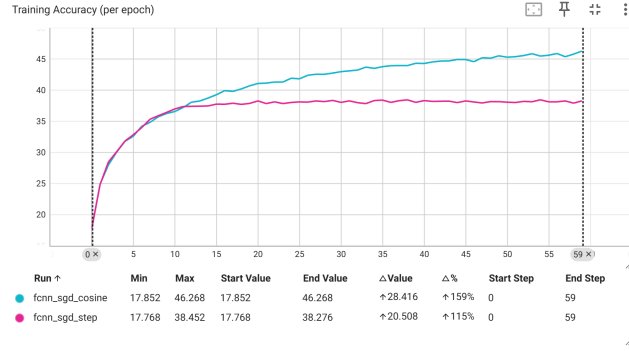


(b) Training Loss

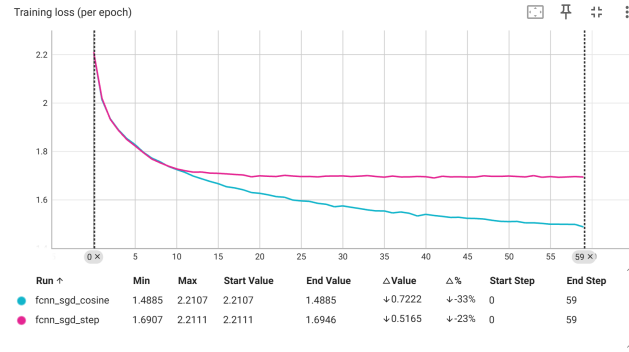


(c) Validation Accuracy

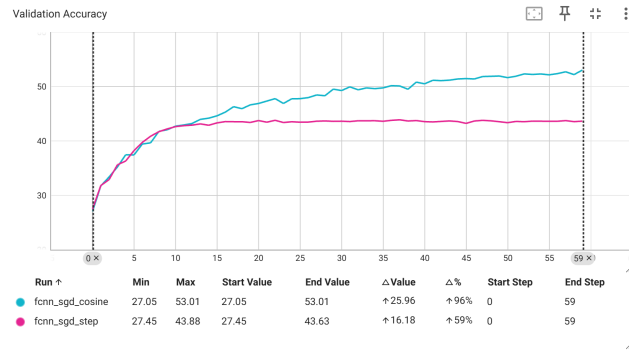
Figure 3: CosineAnnealingLR vs StepLR (AdamW)



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 4: CosineAnnealingLR vs StepLR (SGD)

3.3 Visualization

3.3.1 Linear Classifier

Configurations:

- optimizer: AdamW
- AdamW_lr=0.001
- scheduler: CosineAnnealingLR
- CosineAnnealingLR_T_max=100
- epoch_num=60
- batch_size=128

The test accuracy of our linear classifier is only 38.47%. Poor performance.

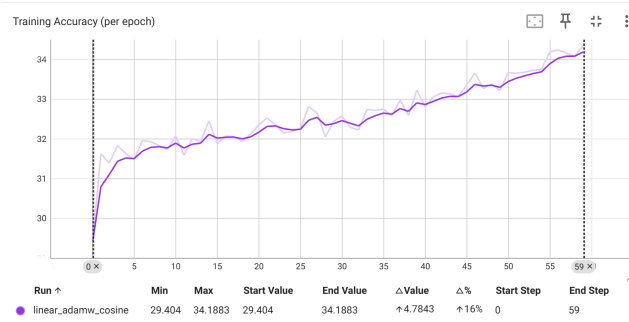
3.3.2 FCNN

Configurations:

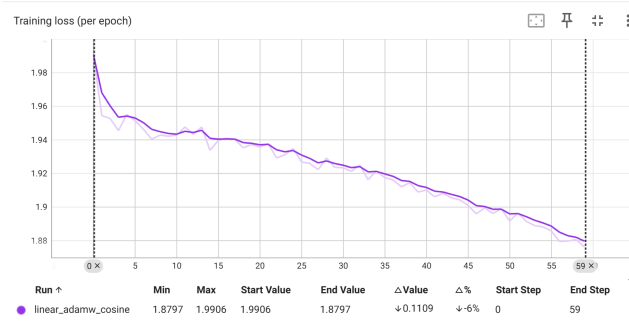
- optimizer: AdamW
- AdamW_lr=0.001
- scheduler: CosineAnnealingLR
- CosineAnnealingLR_T_max=100
- epoch_num=60
- batch_size=128

The test accuracy of our linear classifier is 60.5%. Success!

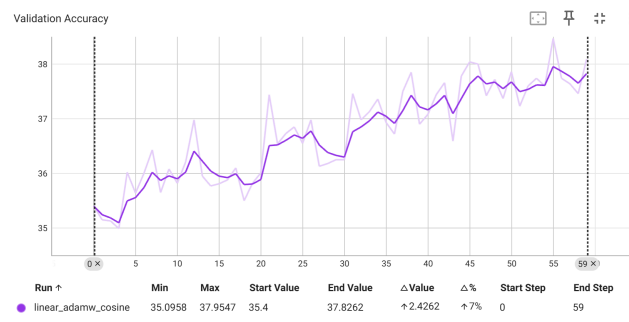
Note: You might have noticed that our accuracies on training data turn out to be lower than those on test data. This is a normal phenomenon because we have used data augmentation to prevent overfitting, which may be a bit excessive; Yet, after all, it is not a bad thing.



(a) Training Accuracy

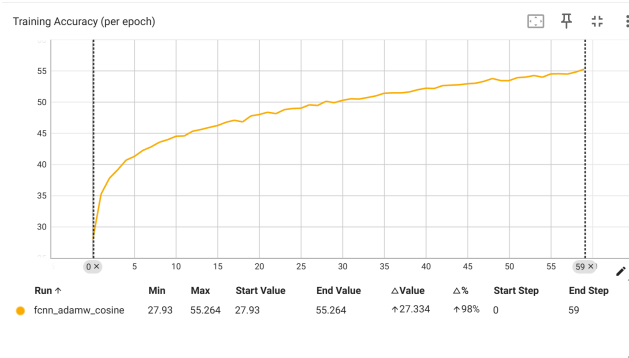


(b) Training Loss

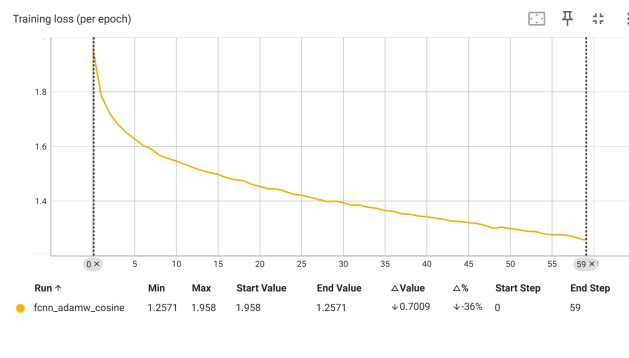


(c) Validation Accuracy

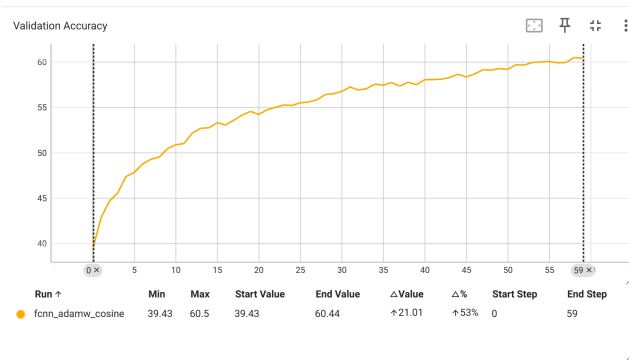
Figure 5: Linear Classifier



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 6: FCNN